Módulo 4 — Introducción a JavaScript

Objetivo: entender qué es JavaScript, cómo se integra con HTML/CSS y dominar los comandos/estructuras más usados para crear interactividad en páginas web.

1. ¿Qué es JavaScript?

JavaScript (JS) es un lenguaje de programación que se ejecuta principalmente en el navegador. Su función típica es añadir lógica e interactividad a páginas web (validar formularios, responder a clics, modificar el contenido, pedir datos a un servidor, etc.).

Puede ejecutarse fuera del navegador (por ejemplo, con Node.js), pero nosotros no vemos esto, sólo páginas web.

JS trabaja junto a:

- HTML: estructura y contenido.
- CSS: aspecto y diseño.
- JavaScript: comportamiento (interacción y lógica).
-

1.1. ¿Dónde se escribe y cómo se ejecuta?

Hay dos formas básicas:

• Dentro de un documento HTML con la etiqueta <script>.

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>Ejemplo</title>
</head>
<body>
  <h1>Hola</h1>
  <script>
    console.log("JS se ejecuta al cargar la página");
  </script>
</body>
</html>
```

• En un archivo externo (recomendado) y enlazado con src:

```
<!--esto calcula el inventario -->

<script src="./scripts/inventario.js"></script>
```

2. Plan de clase

1) Introducción, ejecución en navegador, consola, variables y tipos. Mini ejercicios.
2) Operadores, condicionales (if/switch) y bucles (for/while).
3) Funciones, scope, arrays y métodos comunes.
4) Objetos, JSON y estructuras de datos básicas.
5) DOM: seleccionar elementos, cambiar texto/HTML, clases CSS, crear nodos
6) Eventos (click, input, submit) + mini proyecto guiado: formulario que añade filas a una tabla

3. Comandos y estructuras fundamentales

En JavaScript no se habla tanto de “comandos” como en un terminal, sino de palabras clave, funciones y métodos. A continuación tienes los más usados en la práctica web.

3.1. console.log()

Muestra mensajes en la consola del navegador (F12). Es tu mejor amigo para depurar.

```
console.log("Hola desde la consola");
let x = 10;
console.log("x vale:", x);
```

3.2. let, const y var

Declaran variables. Hoy se recomienda usar let (cambia) y const (no cambia). Evita var salvo casos heredados.

```
let contador = 0;
contador = contador + 1;

const IVA = 0.21;
// IVA = 0.10; // Esto daría error
```

3.3. Tipos básicos: string, number, boolean, null, undefined

JS es de tipado dinámico: una variable puede guardar distintos tipos. Usa typeof para inspeccionar.

```
let nombre = "Ana";       // string
let edad = 30;              // number
let activo = true;          // boolean
let vacio = null;           // null (valor intencionalmente vacío)
let sinValor;               // undefined

console.log(typeof nombre);   // "string"
console.log(typeof edad);     // "number"
console.log(typeof activo);   // "boolean"
console.log(sinValor);        // undefined
```

3.4. Operadores (+, -, *, /, %, **, ==, ===, !=, !==, <, >, &&, ||, !)

Permiten calcular y comparar. Recomendación: usa === y !== para comparar con tipo y valor.

```
console.log(2 + 3);       // 5
console.log(10 % 3);        // 1 (resto)
console.log(2 ** 3);        // 8 (potencia)

console.log(5 == "5");      // true (convierte tipos)  -> evita
console.log(5 === "5");     // false (estricto)        -> recomendado

console.log(true && false); // false
console.log(!true);          // false
```

3.5. if / else

Estructura de decisión. Ejecuta un bloque u otro según una condición.

```
let nota = 6;

if (nota >= 5) {
  console.log("Aprobado");
} else {
  console.log("Suspenso");
}
```

3.6. switch

Alternativa a muchos if/else cuando comparas un valor con varios casos.

```
let dia = "L";

switch (dia) {
  case "L":
    console.log("Lunes");
    break;
```

```
  case "M":
    console.log("Martes");
    break;
  default:
    console.log("Otro");
}
```

3.7. for

Bucle con contador, ideal para repetir N veces.

```
for (let i = 1; i <= 5; i++) {
  console.log("i =", i);
}
```

3.8. while / do…while

Bucle que repite mientras la condición sea verdadera. do...while se ejecuta al menos una vez.

```
let n = 3;
while (n > 0) {
  console.log(n);
  n--;
}

// do...while
let m = 0;
do {
  console.log("m =", m);
  m++;
} while (m < 3);
```

3.9. function (funciones)

Agrupan instrucciones y permiten reutilizar código. Pueden devolver valores con return.

```
function sumar(a, b) {
  return a + b;
}

console.log(sumar(2, 3)); // 5
```

3.10. Funciones flecha (=>)

Sintaxis moderna y muy usada en eventos y callbacks.

```
const sumar = (a, b) => a + b;

const saludar = (nombre) => {
  console.log("Hola", nombre);
};
```

```
console.log(sumar(4, 5));
saludar("Luis");
```

3.11. Arrays (listas) + métodos comunes

Un array guarda una colección ordenada. Métodos típicos: push, pop, includes, indexOf, forEach, map, filter, find, reduce.

```
let ciudades = ["Madrid", "Valencia", "Sevilla"];

ciudades.push("Bilbao");      // añadir al final
console.log(ciudades);

ciudades.pop();               // quitar el último
console.log(ciudades);

ciudades.forEach(c => console.log("Ciudad:", c));

let mayus = ciudades.map(c => c.toUpperCase());
console.log(mayus);

let conM = ciudades.filter(c => c.startsWith("M"));
console.log(conM);
```

3.12. Objetos ({})

Un objeto agrupa datos con propiedades (clave: valor). Muy usados para representar entidades.

```
let persona = {
  nombre: "Ana",
  edad: 30,
  activo: true
};

console.log(persona.nombre);
persona.edad = 31;
```

3.13. JSON.stringify() y JSON.parse()

JSON es el formato típico para intercambiar datos con servidores. stringify convierte objeto->texto; parse texto->objeto.

```
let producto = { id: 1, nombre: "Teclado", precio: 19.99 };

let texto = JSON.stringify(producto);
console.log(texto); // {"id":1,"nombre":"Teclado","precio":19.99}

let otraVezObjeto = JSON.parse(texto);
console.log(otraVezObjeto.nombre);
```

3.14. document.getElementById() / querySelector()

Seleccionan elementos del DOM para poder leer o modificar la página.

```
// HTML: <p id="msg"></p>
const p = document.getElementById("msg");
p.textContent = "Hola desde JS";

// querySelector usa selectores CSS
const primerBoton = document.querySelector("button");
console.log(primerBoton);
```

3.15. textContent vs innerHTML

textContent inserta texto “tal cual”. innerHTML permite insertar HTML (úsalo con cuidado si viene de usuario).

```
// HTML: <div id="caja"></div>
const caja = document.getElementById("caja");

caja.textContent = "<b>Hola</b>";  // se verá literalmente
caja.innerHTML = "<b>Hola</b>";   // se verá en negrita
```

3.16. classList (add/remove/toggle)

Añade o quita clases CSS para cambiar estilos sin tocar directamente CSS en JS.

```
// HTML: <p id="mensaje" class="ok">...</p>
const el = document.getElementById("mensaje");

el.classList.add("resaltado");
el.classList.remove("ok");
el.classList.toggle("oculto");
```

3.17. createElement() + appendChild()

Crea elementos nuevos en la página dinámicamente.

```
// HTML: <ul id="lista"></ul>
const ul = document.getElementById("lista");

const li = document.createElement("li");
li.textContent = "Elemento nuevo";
ul.appendChild(li);
```

3.18. addEventListener() (eventos)

Escucha acciones del usuario: clics, escritura, envío de formularios, etc.

```
// HTML: <button id="btn">Pulsa</button>
const btn = document.getElementById("btn");

btn.addEventListener("click", () => {
  console.log("Has hecho clic");
});
```

3.19. Eventos de formularios (submit) y preventDefault()

En formularios, submit recarga la página por defecto. Con preventDefault() lo evitamos para validar o procesar en JS.

```
// HTML: <form id="frm">...</form>
const frm = document.getElementById("frm");

frm.addEventListener("submit", (e) => {
  e.preventDefault(); // evita recarga
  console.log("Formulario enviado (sin recargar)");
});
```

3.20. Conversión a número: Number(), parseInt(), parseFloat()

Los inputs devuelven texto. Convierte antes de sumar para evitar '2' + '3' = '23'.

```
const a = "2";
const b = "3";

console.log(a + b);              // "23" (concatena)
console.log(Number(a) + Number(b)); // 5

console.log(parseInt("10", 10));   // 10
console.log(parseFloat("3.14"));   // 3.14
```

3.21. setTimeout() y setInterval()

Ejecutan código tras un tiempo o repetidamente (útiles para animaciones sencillas y temporizadores).

```
setTimeout(() => {
  console.log("Han pasado 2 segundos");
}, 2000);

let contador = 0;
const id = setInterval(() => {
  contador++;
  console.log("tick", contador);
  if (contador === 5) clearInterval(id);
}, 1000);
```

3.22. fetch() (peticiones HTTP)

Permite pedir datos a un servidor (APIs). Devuelve una promesa; se suele usar con async/await.

```
async function cargarUsuarios() {
  const resp = await fetch("https://jsonplaceholder.typicode.com/users");
  const data = await resp.json();
  console.log("Usuarios:", data);
}

cargarUsuarios();
```

3.23. localStorage (guardar en el navegador)

Guarda pares clave-valor como texto. Útil para preferencias simples.

```
localStorage.setItem("tema", "oscuro");

const tema = localStorage.getItem("tema");
console.log(tema);

localStorage.removeItem("tema");
```

4. Mini proyecto guiado (práctica)

Objetivo: crear una página con un formulario (nombre, ciudad, sexo) que al enviar añada una fila a una tabla HTML.

Practicamos DOM, eventos, conversiones y validación básica.

4.1. HTML (estructura)

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>Mini proyecto</title>
  <style>
    body { font-family: Arial, sans-serif; padding: 16px; }
    table { border-collapse: collapse; margin-top: 12px; }
    th, td { border: 1px solid #ccc; padding: 8px; }
  </style>
</head>
<body>

  <h1>Alta de alumnos</h1>

  <form id="frm">
    <label>Nombre:
```

```
      <input id="txtNombre" required>
    </label>

    <label>Ciudad:
      <select id="cboCiudad" required>
        <option value="">-- Selecciona --</option>
        <option>Madrid</option>
        <option>Barcelona</option>
        <option>Valencia</option>
        <option>Sevilla</option>
        <option>Bilbao</option>
      </select>
    </label>

    <fieldset>
      <legend>Sexo</legend>
      <label><input type="radio" name="sexo" value="Hombre" required>
Hombre</label>
      <label><input type="radio" name="sexo" value="Mujer"> Mujer</label>
    </fieldset>

    <button type="submit">Añadir</button>
  </form>

  <table>
    <thead>
      <tr><th>Nombre</th><th>Ciudad</th><th>Sexo</th></tr>
    </thead>
    <tbody id="tbody"></tbody>
  </table>

  <script src="app.js"></script>
</body>
</html>
```

4.2. JavaScript (app.js)

```
const frm = document.getElementById("frm");
const txtNombre = document.getElementById("txtNombre");
const cboCiudad = document.getElementById("cboCiudad");
const tbody = document.getElementById("tbody");

frm.addEventListener("submit", (e) => {
  e.preventDefault();

  // 1) Leer valores
  const nombre = txtNombre.value.trim();
  const ciudad = cboCiudad.value;
  const sexo = document.querySelector("input[name='sexo']:checked")?.value;

  // 2) Validación mínima
  if (!nombre || !ciudad || !sexo) {
    // Podrías mostrar el error en pantalla en vez de alert
```

```
    console.log("Faltan datos");
    return;
  }



  // 3) Crear fila y celdas
  const tr = document.createElement("tr");

  const tdNombre = document.createElement("td");
  tdNombre.textContent = nombre;

  const tdCiudad = document.createElement("td");
  tdCiudad.textContent = ciudad;

  const tdSexo = document.createElement("td");
  tdSexo.textContent = sexo;

  tr.appendChild(tdNombre);
  tr.appendChild(tdCiudad);
  tr.appendChild(tdSexo);

  // 4) Añadir al tbody
  tbody.appendChild(tr);

  // 5) Limpiar formulario
  frm.reset();
  txtNombre.focus();
});
```

5. Ejercicios propuestos (para clase y casa)

1. 1) Calculadora: dos inputs numéricos + botones (sumar, restar, multiplicar, dividir) y resultado en un <p>.
2. 2) Lista de tareas: input + botón “Añadir” que cree <li>; botón para borrar cada tarea.
3. 3) Validación de formulario: email con expresión regular sencilla + mensaje de error en un .
4. 4) Filtrado: array de ciudades, mostrar en <ul> y filtrar según texto escrito en un input (evento input).
5. 5) Mini-consumo de API: usar fetch para traer un listado (por ejemplo usuarios) y pintarlos en pantalla.

6. Resumen

- Depuración: console.log(), consola (F12).
- Variables: const (no cambia), let (cambia).
- Comparación recomendada: === y !==.
- DOM: document.getElementById(), querySelector(), textContent/innerHTML.
- Eventos: addEventListener('click'/'input'/'submit').
- Crear nodos: createElement(), appendChild().
- Arrays: push/pop, forEach, map, filter.
- JSON: JSON.stringify() / JSON.parse().